



REVERIE

Sequential Movie & TV Recommender

A GRU-based next-item prediction model trained on MovieLens ml-1m.

IE University • Deep Learning • July 2026

Amaly Attia • Em Echeverria • Lea Sarouphim Hochar • Stephan Pentchev • Cécile Tambey

Most recommenders average your whole history

**Taste is a
sequence,
not an
average.**

*What you watched last week matters more
than what you watched 3 years ago.*



Recommendations feel stale

- Netflix / Spotify average your entire history, so the homepage stops feeling fresh.



A recurrent model reads order

Reverie reads your history in order and predicts what comes next.



It corrects itself instantly

- Reject a recommendation and Reverie re-ranks on the spot — no retraining.

MovieLens ml-1m, split leave-one-out

1,000,209

RATINGS

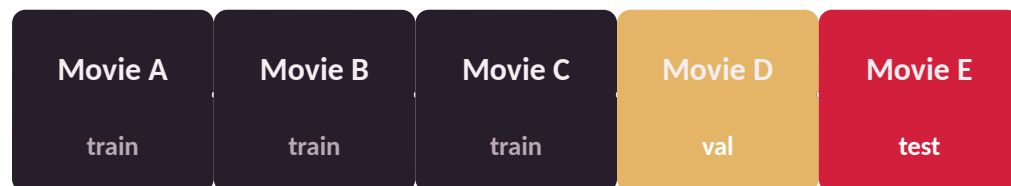
6,040

USERS

3,413

MOVIES

Leave-one-out, by timestamp



The test movie is sealed until the frozen config — the model never sees it during training or tuning.



Why ml-1m?

- 10× more sequences than ml-latest-small (100K).
- A reliable softmax over thousands of movies needs dense data.
- Movies seen < 5 times filtered out → 3,413 kept.

Hybrid GRU



Movie embedding 32d Each movie ID → 32 learned numbers; similar movies sit close together.	Genre lookup (frozen) Appends genre tags (Action, Drama...) — fixed, not trained.	Star-rating channel Appends your rescaled 1-5 rating — loved it or hated it.
GRU 64 units Reads the sequence left-to-right into one taste vector.	Dropout 0.3 Zeroes 30% of neurons in training — prevents memorization.	Softmax A probability for every movie — highest = recommendation.

Experiments E3 / E4 / E5 — nothing guessed

E3 — history length

Length	HR@10
L = 10	0.2707
L = 20	0.2779
L = 50	0.2739

L=20 beats L=50 — recent taste > old history.

E4 — model size

dim / units	HR@10
16 / 64	0.2725
32 / 64	0.2740
32 / 128	0.2779
64 / 128	0.2782
64 / 256	0.2687

E5 — cell type

Cell	HR@10
GRU	0.2740
LSTM	0.2730

Tied within 0.001 — GRU wins on fewer parameters.

Locked config: history 20 • embed 32 • 64 GRU units • GRU cell

We picked the smallest config within noise of the best — bigger isn't better (d64/u256 overfits). Every choice is an empirical result on the validation set.

Experiment E6 — feature ablation

Input variant	HR@10	MRR	NDCG@10
Movie ID only	0.2765	0.1310	0.1539
+ Genre (frozen)	0.2812	0.1324	0.1561
+ Rating only	0.2558	0.1290	0.1471
Full hybrid	0.2740	0.1345	0.1555



Decision: full hybrid

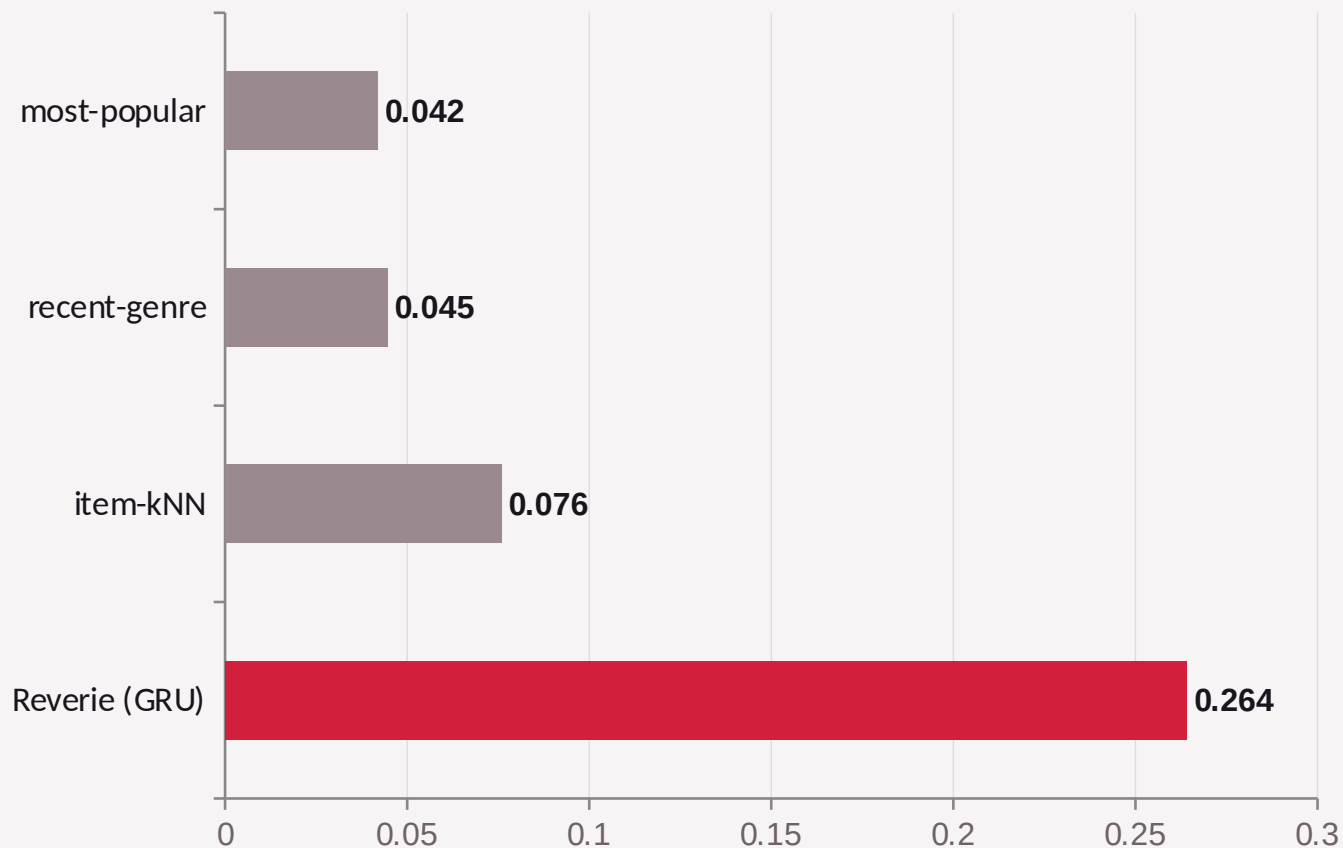
- Best MRR (0.1345) — ranks the right movie highest.
- Best NDCG — best overall ranking quality.
- Rating channel is required for the thumbs-down feature.



Why does +rating alone hurt HR@10?

Star ratings are noisy on their own — a user might give 5 stars to both an action film and a romance. Without genre context the model can't tell which signal to follow. Combined with genre, the rating becomes meaningful.

Experiment E0 — GRU vs non-neural methods



6× better than popular

- GRU 0.264 vs most-popular 0.042 — and it beats item-kNN, which shallow models often lose to.



Statistically proven

- Non-overlapping 95% CIs; Wilcoxon $p \approx 0$ vs all three baselines. The win is not luck.

The headline number

HR@10 = 0.242

± 0.002 (mean $\pm$ std, 3 seeds, blind test set)

In plain English

The model puts the movie a user actually watched next inside its top-10 recommendations 24.2% of the time — across 6,032 users, on data it never saw.



Blind test set

Sealed from day 1; never seen during E3/E4/E5 tuning.



3 seeds (42, 0, 7)

One run is random — 3 seeds show the result is stable.



Full-catalog ranking

All 3,413 movies ranked, not 100 sampled negatives.



LIVE DEMO

Reverie in action



Add a few movies with star ratings to build a history



Recommendation row updates in real time



Thumbs-down a pick → it disappears and the list re-ranks live



Taste-profile panel shows the genre breakdown

Two terminals

Terminal 1

```
uvicorn app.api:app  
--port 8000
```

Terminal 2

```
cd web  
npm run dev
```

Open

localhost:5173

Backup recording ready in case the live demo breaks.

Honest about the edges



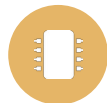
Movie-only training, TV via proxy

TV recs use a genre-similarity bridge — not a trained signal. An approximation.



Cold start

A new user with no history gets generic picks; needs 3–4 ratings to personalize.



CPU training only (Windows)

TF dropped Windows GPU after 2.10; ~13s/epoch on CPU limited how many runs we could do.



Single softmax over 3,413 movies

Correct and rigorous, but doesn't scale to 100M-item catalogs without retrieve-then-rank.



No explicit cross-user signal

Collaborative signals are implicit via shared embeddings, never stated outright.

What we'd do next — not failures



E2 — Logit adjustment (popularity-bias correction) *one line of code*

The softmax over-recommends blockbusters that appear disproportionately in training. Subtracting the log of each movie's frequency surfaces niche picks. We held off because the model already beats every baseline and we didn't want to change output right before the test run — but it's the most impactful next improvement.



E1 — Dataset comparison

Formally quantify ml-1m vs ml-latest-small (we chose ml-1m on rationale).



Multi-layer GRU

Stack a second GRU layer — likely helps on larger datasets.



SASRec / Transformer

Attention models win at scale (millions of users); the natural next step.



GPU training via WSL2

~6-7× faster epochs → more experiments, bigger models.



Reverie



It works — $\text{HR@10} = 0.242 \pm 0.002$ on a blind test set. 6× better than most-popular; beats item-kNN.



Every decision was empirical — L=20 (E3), 32/64 (E4), GRU (E5), full hybrid (E6) — all justified by validation numbers.



It runs — React frontend + FastAPI backend + a trained model serving real-time recs. The loop closes.